GitLab — Zero to Master

21 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction to GitLab

02 Accounts, Groups and Projects

03 SSH Keys and Cloning

04 Issues, Labels and Milestones

05 Merge Requests

06 Web IDE and Wiki

Intermediate (6 topics)

07 Branch Protection and Approvals

08 Introduction to CI/CD and .gitlab-ci.yml

09 Runners and Executors

10 Variables and Secrets

11 Artifacts and Caching

12 Environments and Deployments

Advanced (6 topics)

13 Advanced Pipelines

14 Container Registry and Docker Builds

15 Security Scanning

16 Package Registry and Releases

17 GitLab Pages

18 Kubernetes Integration and GitOps

Master (3 topics)

19	Self-managed GitLab Administration
20	Pipeline Optimisation and Governance
21	Real Project: Full CI/CD Pipeline

Beginner Level

01. Introduction to GitLab

GitLab is a DevOps platform: Git hosting, issues, merge requests, CI/CD, container registry and security scanning in one product (SaaS or self-managed).

02. Accounts, Groups and Projects

Create an account, groups (teams) and projects (repositories); set visibility and roles (Guest, Reporter, Developer, Maintainer, Owner).

03. SSH Keys and Cloning

Add an SSH key to your profile and clone with SSH or HTTPS.

```
ssh-keygen -t ed25519
git clone git@gitlab.com:group/project.git
```

04. Issues, Labels and Milestones

Track tasks and bugs; use labels, boards, milestones and weights.

05. Merge Requests

Propose changes from a branch, request review, discuss and merge.

```
git push -o merge_request.create origin feature/x
```

06. Web IDE and Wiki

Edit files in the browser; document with the project wiki.

Intermediate Level

07. Branch Protection and Approvals

Protect `main`, require approvals, code owners and pipelines to pass before merge.

08. Introduction to CI/CD and .gitlab-ci.yml

Pipelines made of stages and jobs defined in `.gitlab-ci.yml`; runners execute jobs.

```yaml
stages: [build, test]
build-job:
  stage: build
  script:
    - npm ci
    - npm run build
```

09. Runners and Executors

Shared vs project runners; Docker, shell and Kubernetes executors; register a runner.

```
gitlab-runner register
```

10. Variables and Secrets

CI/CD variables (masked, protected, file); predefined variables like `$CI_COMMIT_SHA`.

```yaml
script:
  - echo $CI_COMMIT_BRANCH
```

11. Artifacts and Caching

Pass build outputs between jobs and speed up with caches.

```yaml
artifacts:
  paths: [dist/]
cache:
  paths: [node_modules/]
```

12. Environments and Deployments

Deploy jobs with environments, manual approvals and review apps.

```yaml
deploy:
  stage: deploy
  environment: production
  when: manual
```

Advanced Level

13. Advanced Pipelines

`rules`, `needs` (DAG), `include`, `extends`, templates, parent-child and multi-project pipelines.

```
test:
  needs: [build]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

14. Container Registry and Docker Builds

Build and push images to the GitLab registry in CI.

```
script:
  - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
  - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

15. Security Scanning

SAST, dependency, secret and container scanning, DAST via templates.

```
include:
  - template: Security/SAST.gitlab-ci.yml
```

16. Package Registry and Releases

Publish npm, Maven, PyPI packages; create releases with notes and assets.

17. GitLab Pages

Host static sites from a `pages` job.

```
pages:
  script: [mkdir public, cp -r dist/* public]
  artifacts: { paths: [public] }
```

18. Kubernetes Integration and GitOps

Deploy to clusters using the GitLab agent for Kubernetes and GitOps flows.

Master Level

19. Self-managed GitLab Administration

Install via Omnibus/Helm, backups, upgrades, LDAP/SSO, runners at scale, monitoring.

```
sudo gitlab-backup create
```

20. Pipeline Optimisation and Governance

Fast pipelines, compliance pipelines, merge trains, protected environments, audit events.

21. Real Project: Full CI/CD Pipeline

Build -> test -> scan -> containerise -> deploy a web app to staging and production with approvals.

Interview Question Bank

Q1. GitLab vs GitHub?
Both host Git; GitLab bundles a complete DevOps platform with built-in CI/CD, registry and security, and can be self-hosted.

Q2. What is a GitLab Runner?
An agent that executes CI/CD jobs on a machine/container.

Q3. What is `.gitlab-ci.yml`?
The file defining pipeline stages, jobs and rules.

Q4. Stage vs job?
A job is a unit of work; stages group jobs that run in order, jobs in a stage run in parallel.

Q5. Artifacts vs cache?
Artifacts are job outputs passed on; cache stores dependencies to speed up repeated jobs.

Q6. What is a merge request?
A request to merge a source branch into a target with review and CI checks.

Q7. How do you keep secrets safe?
Masked/protected CI variables or external secret managers; never commit them.

Q8. What are protected branches?
Branches with restricted push/merge rights.

Q9. What is `rules:` vs `only/except`?
`rules` is the modern, more flexible way to control when jobs run.

Q10. What is GitLab Pages?
Static site hosting directly from a repository.

Capstone Project Ideas

1. CI pipeline for a Node app
2. Docker build and push to registry
3. Deploy to a VPS with environments
4. Publish a static site with Pages
5. Security scanning pipeline